

Blockbuster+ - 8-Minute Presentation Script

Joshua Ojeda | Mini Labs 1, 2, and 3 | Estimated speaking time: 7:45 to 8:15

Before You Begin

- Open <http://localhost:3000> on the Dashboard.
- Keep <http://localhost:3000/about> and <http://localhost:3000/api-docs/> ready in separate tabs.
- Keep the Catalog Manager at <http://localhost:3000/catalog-manager> ready for the CRUD demo.
- Take one breath before starting. Speak slower than feels natural.

BACKUP LINE

If anything takes a second to load, say: "This is live data coming through my own API, so I will give it one second. The app also has loading, error, retry, and offline fallback states."

0:00-0:45 - Opening

ON SCREEN

Start on the Blockbuster+ Dashboard. Do not click immediately. Let the class see the page while you introduce it.

Hey everyone. So, I am a little nervous, but I am excited to show you Blockbuster+. This is a movie discovery and rental-planning app that I built across Mini Labs 1, 2, and 3. Instead of making three disconnected assignments, I kept evolving the same idea. It started as a responsive front-end storefront, became a React application, and then became a full-stack MVC project with its own API and SQLite database. The visual idea is a digital version of walking through an old video store, with movie aisles, VHS-style cases, ratings, and a rental bag.

0:45-1:25 - One Project, Three Labs

ON SCREEN

Open the About page and point to the three Mini Lab sections.

The About page is also my simple grading map. Mini Lab 1 created the storefront and the original interactions. Mini Lab 2 rebuilt that experience with React, Vite, reusable components, routing, hooks, and shared state. Mini Lab 3 added the Express backend, MVC structure, startup data import, database, complete CRUD, Swagger, and testing. They are separate stages, but the final result is one application, one codebase, one API, and one database-backed film catalog.

1:25-2:15 - Mini Lab 1: The Storefront Foundation

ON SCREEN

Return to the Dashboard, then click Browse Aisles.

Mini Lab 1 is the foundation of the user experience. The original version used semantic HTML, Bootstrap, JavaScript, Fetch, and local JSON to create a responsive three-page movie storefront. I kept the required search, genre filtering, dynamic movie cards, ratings, review elements, and rental-bag idea. I also kept the original Bootstrap version as a snapshot, so the progression is honest. In the final app, those same ideas are more reusable and accessible, but the main goal is still simple: help somebody browse films and choose what they want to watch.

2:15-3:25 - Mini Lab 2: React and Vite

ON SCREEN

On Films, search for a title, change a genre or sort option, then open one movie card.

Mini Lab 2 is where the site became a real React application. The Vite frontend lives in the apps/web workspace. Every screen uses functional components, and React Router handles the Dashboard, Films, Film Details, Rental Bag, Catalog Manager, and About routes without full page reloads. The catalog is not a set of hardcoded cards. A custom useFilms hook uses Axios to request records from my API, while useEffect manages the request lifecycle. The page also has loading, error, empty, retry, and success states, so it does not assume the network always behaves perfectly.

Search, filters, sorting, forms, and dialogs use useState. Derived catalog results use useMemo instead of changing the original array. Every rendered film has a stable database ID as its React key. The URL remembers catalog filters, which means a filtered view can be refreshed or shared without losing its state. Material UI provides responsive controls, consistent spacing, and keyboard-visible focus states.

3:25-4:15 - Shared State and Interaction

ON SCREEN

Add the open film to the Rental Bag. If time allows, submit the controlled shelf-note or review form. Then open Rental Bag from the navigation.

The rental bag demonstrates shared application state. I used React Context so a film can be added or removed from different routes without prop drilling. The updates are immutable, and localStorage preserves the selection after navigation or refresh. The review area is a controlled form, so React owns the input values and submission behavior. These details are small, but they show that the app is not only styled to look interactive. The state actually moves through the application in a predictable way.

BACKUP LINE

If a search or filter is already active, click Reset before continuing. If the bag already contains the film, remove it first and add it again.

4:15-5:25 - Mini Lab 3: MVC, API, and Database

ON SCREEN

Open the About page's Mini Lab 3 section, or briefly show the project folders in VS Code: routes, controllers, models, services, config, and apps/web.

Mini Lab 3 is the full-stack layer behind everything I just showed. The Express server follows MVC. Routes match an endpoint to a controller. Controllers handle the HTTP request and response. The Film model owns validation, SQL queries, transactions, and database mapping. A service handles external data seeding, and the React app is the view layer. Keeping those responsibilities separate makes the code easier to test and change.

On the first startup, the app checks whether SQLite is empty. If it is, the seed service fetches the keyless Studio Ghibli API, maps that response into my film data contract, combines it with the curated Blockbuster+ catalog, removes duplicates, and inserts the records. The database schema reflects the incoming movie fields, while repeated data like tags and reviews uses related tables. If the external service is unavailable, a curated fallback still starts the app, which is useful for grading and for a live presentation like this one.

5:25-6:35 - Complete CRUD

ON SCREEN

Open Catalog Manager. Create a temporary film using: Class Demo Movie, Drama, Joshua Ojeda, 2026, 8.7, 102 minutes, and the description "A temporary record created during the Blockbuster+ presentation." Leave Poster URL blank.

The Catalog Manager demonstrates all four CRUD operations against the SQLite database. Creating this record sends a POST request. The database records below are the READ operation. I can click Edit, change the rating to 9, and save, which sends a PATCH request. Then I can click Delete, review the confirmation dialog, and remove it with a DELETE request. The form is controlled in React, but every action still travels through the Express route, controller, model, and database. Server-side validation also checks the required fields and valid ranges instead of trusting the browser.

ON SCREEN

After creating the film, point to its new record. Edit the rating to 9.0, save, then delete it and confirm the dialog.

BACKUP LINE

If you are short on time, create the record and explain the Edit and Delete buttons instead of performing both. The automated tests already execute the full CRUD cycle.

6:35-7:15 - Swagger and API Demonstration

ON SCREEN

Open <http://localhost:3000/api-docs/>. Expand the Films section and point to GET, POST, PATCH, and DELETE.

Because Mini Lab 3 does not require a frontend, I also documented the REST API with Swagger and included a Postman collection. Swagger lets the professor inspect the request schemas and call the same GET, POST, PATCH, and DELETE endpoints directly. The API uses predictable status codes and JSON responses, so the project can be graded from the interface or independently from the browser UI.

7:15-8:00 - Quality Check and Closing

ON SCREEN

Return to the Dashboard. If useful, show a terminal or screenshot with `npm run check` passing.

Before presenting, I ran the full quality command. It lints the Express code and React code, creates a production Vite build, validates the seed catalog, and runs ten automated tests, including API integration, immutable frontend helpers, offline seeding, and a real create-read-update-delete cycle. The current database has about 38 films from the external and curated sources.

So, the short version is that Mini Lab 1 gave Blockbuster+ its storefront and personality, Mini Lab 2 made it a reusable React experience, and Mini Lab 3 made it a complete full-stack application. The part I am most proud of is that I did not just place the assignments beside each other. I connected them into one system that can be browsed, tested, managed, and demonstrated from end to end. Thank you. I am happy to answer questions.

Quick Q&A Cheat Sheet

Why did you combine the labs?

Each assignment was a stage of the same product, so combining them shows how a static idea can evolve into a reusable React frontend and then a database-backed full-stack system.

Why MVC?

MVC separates request routing, HTTP decisions, and database logic. That makes each layer easier to understand, test, and replace.

Why SQLite?	It is relational, persistent, easy to run locally, and does not require the professor to configure a separate database server.
What happens if the external API fails?	The startup service catches the failure and loads the curated fallback catalog, so the application remains demonstrable offline.
Where is React state used?	Local state handles search, filters, forms, reviews, and dialogs. Context handles the shared rental bag, and localStorage preserves it across refreshes.
How do you prove CRUD works?	Catalog Manager performs the full workflow visually, Swagger and Postman expose the same endpoints, and an automated integration test performs a complete CRUD cycle.
What would you add next?	Authentication, role-based catalog permissions, persisted user reviews, and a production-hosted database.

Thirty-Second Recovery Version

Blockbuster+ combines all three mini labs into one movie application. Mini Lab 1 created the responsive storefront, search, filters, ratings, and rental-bag idea. Mini Lab 2 rebuilt it with React, Vite, Router, hooks, Context, Material UI, Axios, and reusable components. Mini Lab 3 added an Express MVC API, startup data from an external API, a normalized SQLite database, complete CRUD, Swagger, Postman, and automated tests. The result is one project that works from the browser all the way through the API to the database.